DTS103TC
Data Structure and Algorithms

Lab 3: Elementary Data Structures

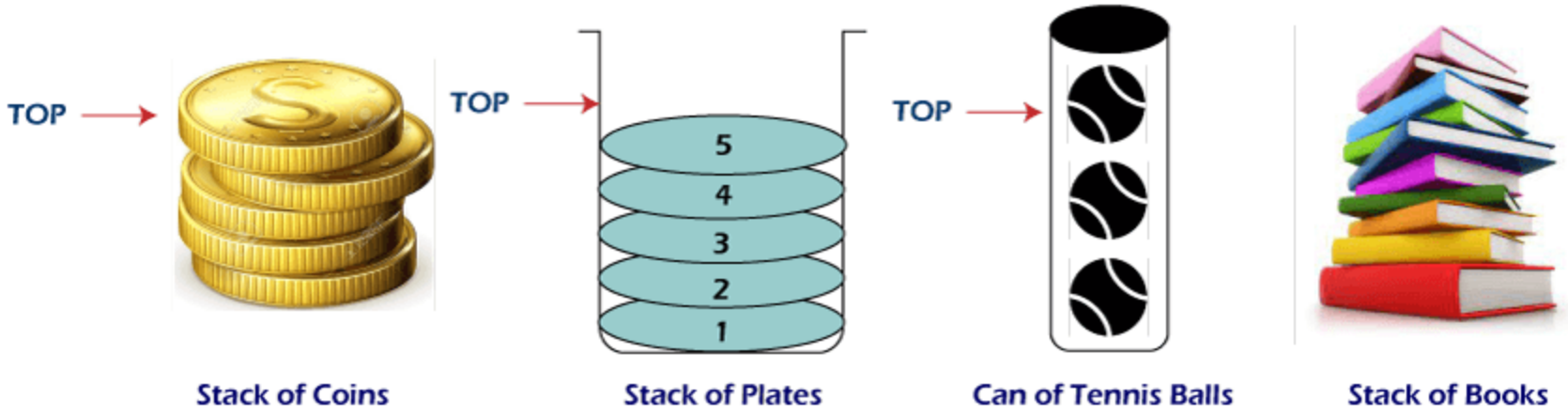
Dr. Qi Chen, Dr. Di Zhang, Dr. Md Maruf Hasan,
Dr. Xuechen Liu, Dr. Guangyu Ren and Dr. Bin Chen
School of AI and Advanced Computing

Learning outcomes

- Implementation of Data Structures
  - Stack
  - Queue
  - LinkedList
  - Heap and Heapsort

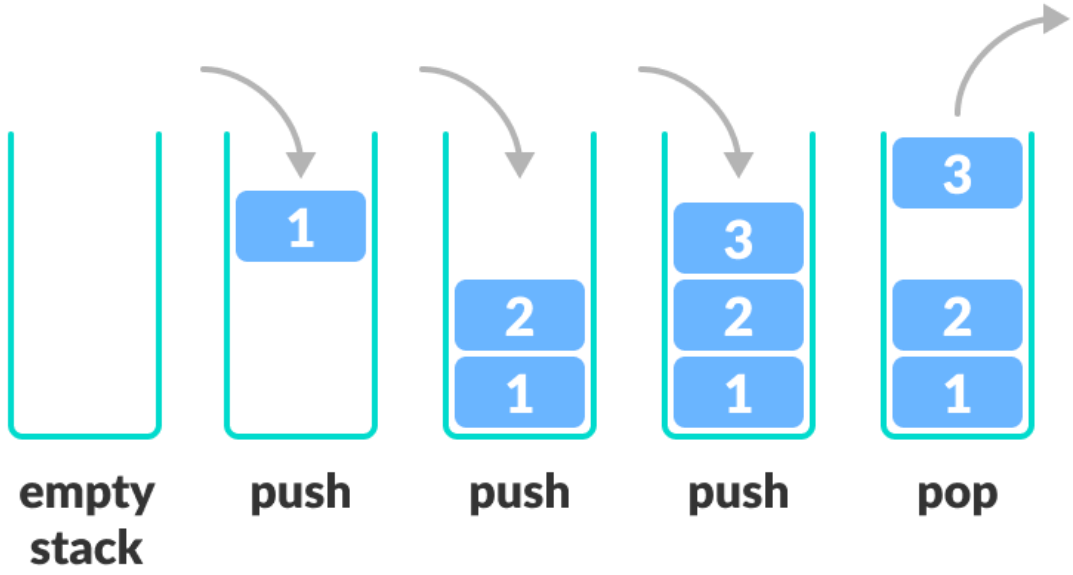
Stack

- A stack is a linear data structure that follows the principle of **Last In First Out (LIFO).**
  - the last element inserted inside the stack is removed first.



Basic Operations of Stack

- **Push:** Add an element to the top of a stack
- **Pop:** Remove an element from the top of a stack
- **Peek:** Get the value of the top element without removing it
- **IsEmpty:** Check if the stack is empty



Stack implementation – Python

- Python lists provide built-in methods that make them suitable for stack operations.

```
class Stack:

    def __init__(self):
        self.stack = []

    # check empty
    def isEmpty(self):
        return len(self.stack) == 0

    # Adding items into the stack
    def push(self, item):
        self.stack.append(item)
        print("pushed item: " + item)

    # Removing an element from the stack
    def pop(self):
        if (self.isEmpty()):
            return "underflow"
        return self.stack.pop()

    # Display the stack
    def display(self):
        print(self.stack)
```

Stack implementation – Python

```
stack = Stack()
stack.push(str(1))
stack.push(str(2))
stack.push(str(3))
stack.push(str(4))
print("popped item: " + stack.pop())
print("stack after popping an element: ")
stack.display()
```

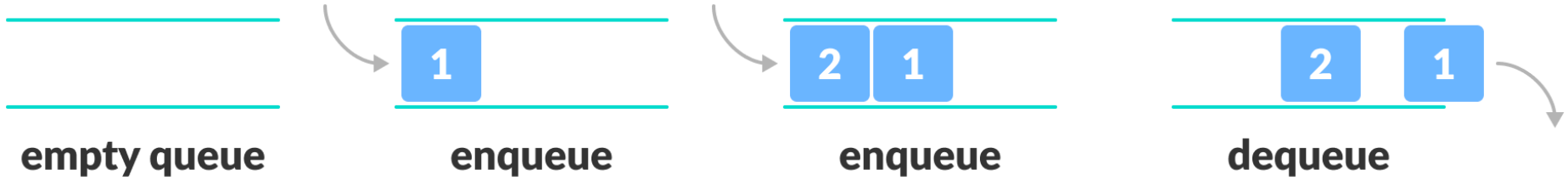
```
pushed item: 1
pushed item: 2
pushed item: 3
pushed item: 4
popped item: 4
stack after popping an element:
['1', '2', '3']
```

Queue

- Queue follows the First In First Out (FIFO) rule
  - the item that goes in first is the item that comes out first.

Basic Operations of Queue

- Enqueue: Add an element to the end of the queue
- Dequeue: Remove an element from the front of the queue
- IsEmpty: Check if the queue is empty
- Peek: Get the value of the front of the queue without removing it



Queue implementation – Python

- Python lists provide built-in methods that make them suitable for queue operations.

```
class Queue:

    def __init__(self):
        self.queue = []

    # Add an element
    def enqueue(self, item):
        self.queue.append(item)

    # Remove an element
    def dequeue(self):
        if (self.isEmpty()):
            return "underflow"
        return self.queue.pop(0)
```

```
def isEmpty(self):
        return len(self.queue) == 0

    # Display the queue
    def display(self):
        print(self.queue)
```

list.pop(0) is O(n)
- Must shift n-1 elements after removing the first element

Python collections.deque supports O(1) operations for inserting and removing elements at both ends

Queue Implementation - Python

```
q = Queue()
q.enqueue(1)
q.enqueue(2)
q.enqueue(3)
q.enqueue(4)
q.display()
q.dequeue()
print("After removing an element")
q.display()
```

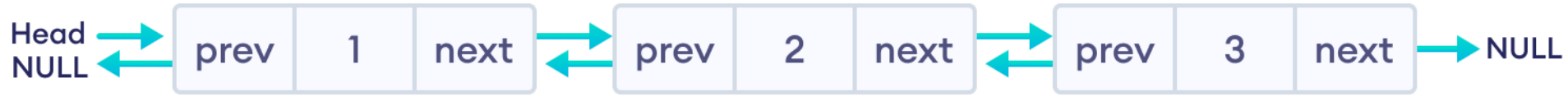
```
[1, 2, 3, 4]
After removing an element
[2, 3, 4]
```

Linked List

- A linked list is a linear data structure that includes a series of connected nodes.
- Unlike an array, the order in a linked list is determined by a pointer in each object.
- Linked lists can be of multiple types: singly, doubly, and circular linked list.

Doubly Linked List

- A doubly linked list is a type of linked list in which each node consists of 3 components:
  - prev - address of the previous node
  - data - data item
  - next - address of next node



- head points to the first node of the linked list.
- next pointer of the last node is NULL.

Operations of Linked List

- Traversal - access each element of the linked list
- Insertion - adds a new element to the linked list
- Deletion - removes the existing elements
- Search - find a node in the linked list

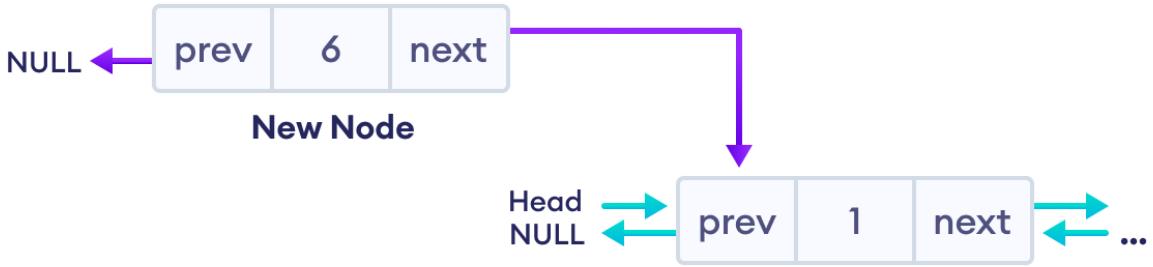
Insertion on a Doubly Linked List

- We can insert elements at 3 different positions of a doubly-linked list:
  - Insertion at the beginning
  - Insertion in-between nodes
  - Insertion at the End

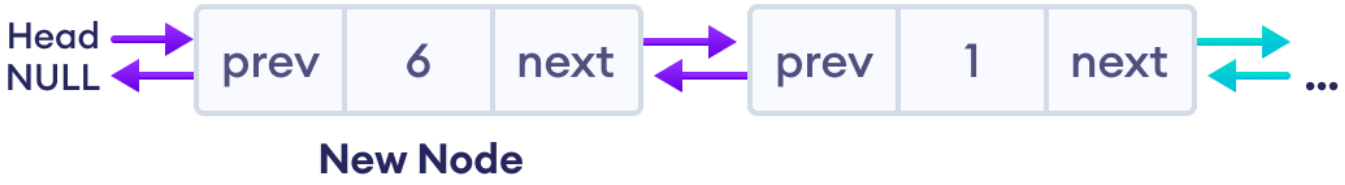
Insertion at the Beginning

- 1. Create a new node

- 2. Set prev and next pointers of new node



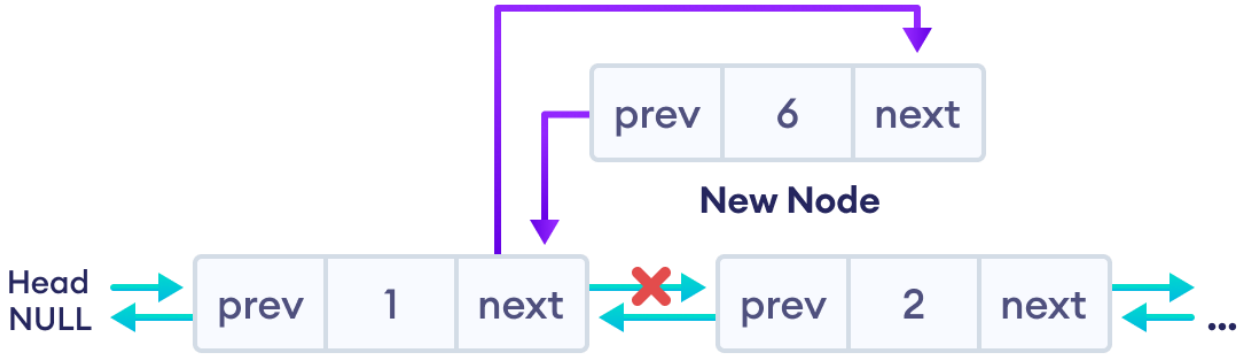
- 3. Make new node as head node



Insertion in between two nodes

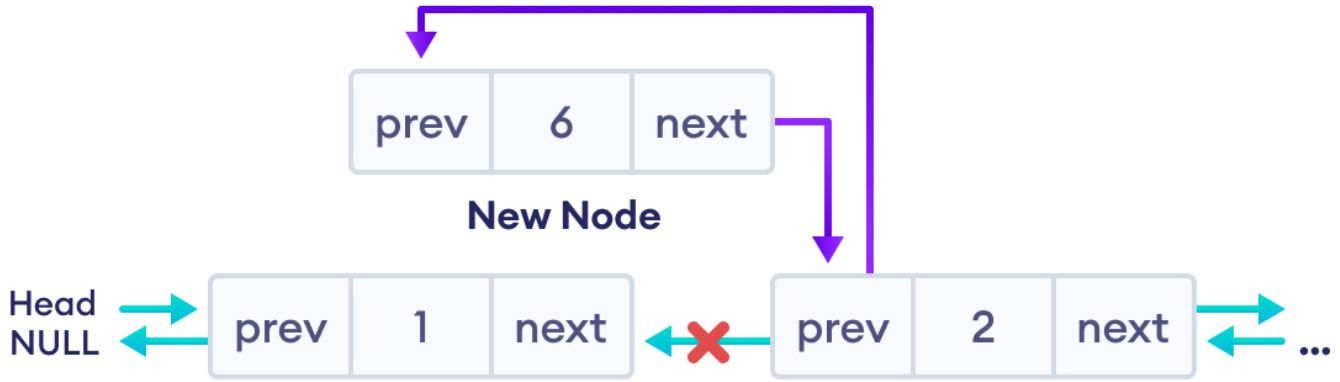
- 1. Create a new node

- 2. Set the next pointer of new node and previous node



Insertion in between two nodes

- 3. Set the prev pointer of new node and the next node

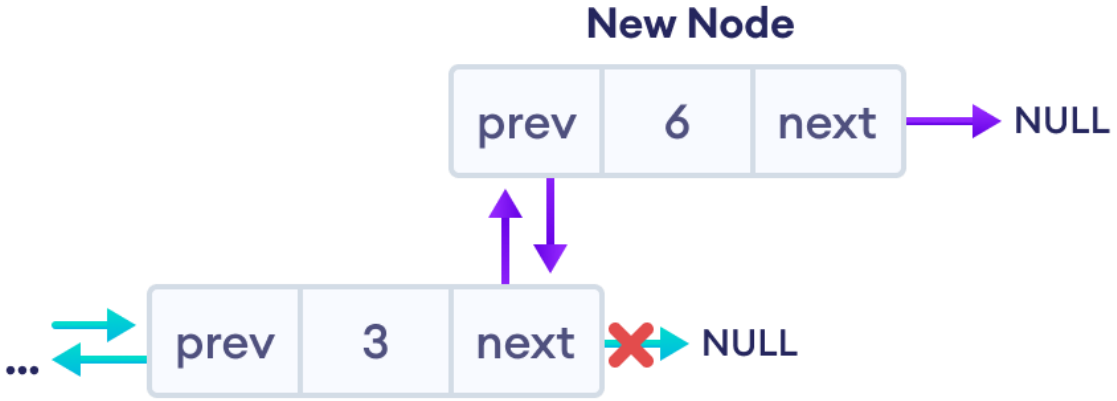


- The final doubly linked list is after this insertion is:

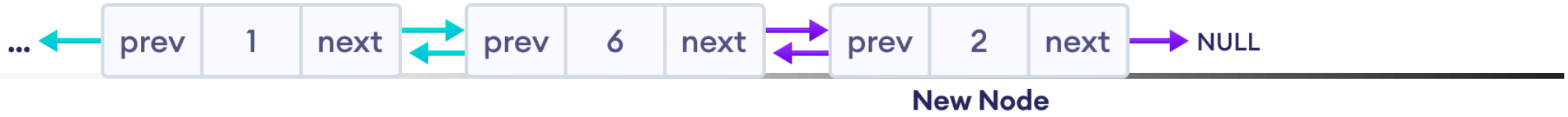
Insertion at the End

- 1. Create a new node

- 2. Set prev and next pointers of new node and the previous node



- The final doubly linked list looks like this.

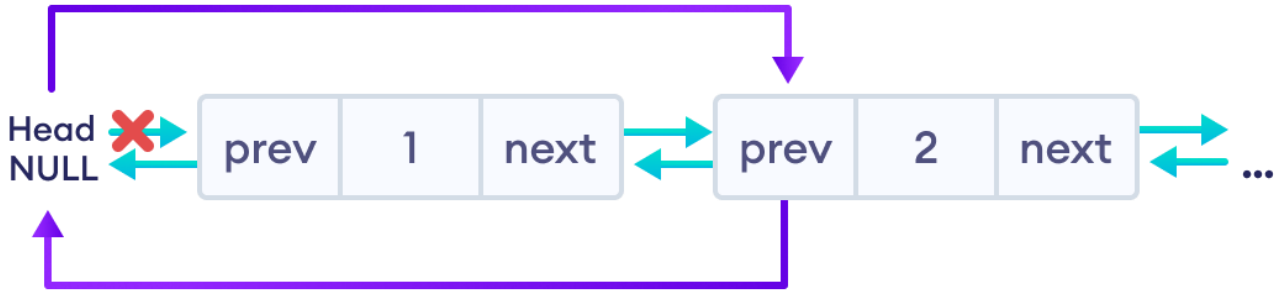


Deletion from a Doubly Linked List

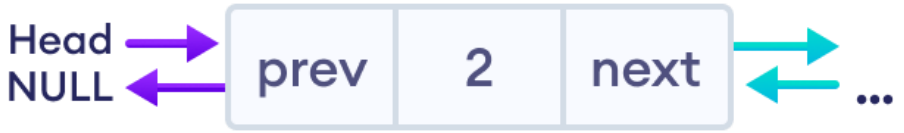
- Similar to insertion, we can also delete a node from 3 different positions of a doubly linked list.
  - 1. Delete the First Node of Doubly Linked List
  - 2. Deletion of the Inner Node
  - 3. Delete the Last Node of Doubly Linked List

Delete the First Node of Doubly Linked List

- If the node to be deleted (i.e. del_node) is at the beginning.
  - Reset value node after the del_node (i.e. node two)



  - Free the memory of del_node, and the linked list will look like this.

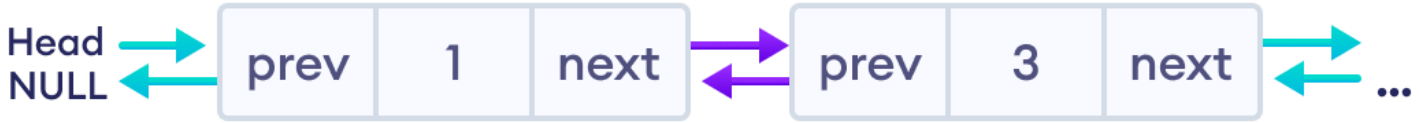


Free the space of the first node

Deletion of the Inner Node

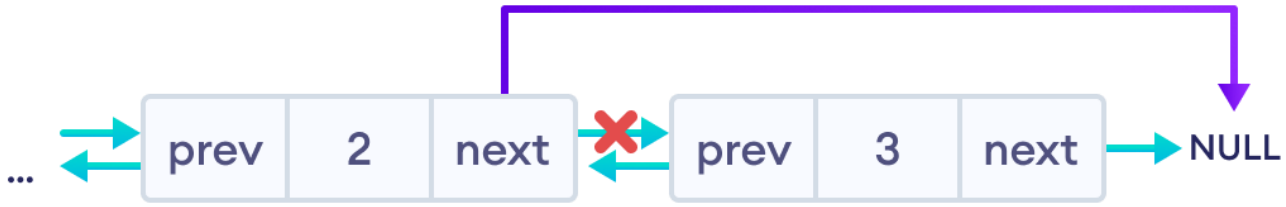
- If del_node is an inner node
  - reset the value of next and prev of the nodes before and after the del_node.

  - Free the memory of del_node, and the linked list will look like this.

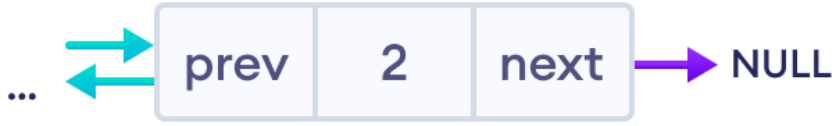


Delete the Last Node of Doubly Linked List

- If the node to be deleted (i.e. del_node) is at the End.
  - simply delete the del_node and make the next of node before del_node point to NULL.



  - Free the memory of del_node, and the linked list will look like this.



Lab Exercises

- Implementation of Data Structures
  - Stack
  - Queue
  - LinkedList
  - Heap and Heapsort (Refer to Lecture Slides)